Pixarch: On-Device Photo Retrieval Pipeline

Project Mid Report

Team: Paisaan

Shivam Patel (2025202030) **Suyash Pande** (2025201063) **Kavan Gandhi** (2025201078)

1 Introduction & Project Overview

Pixarch is an on-device personal data retrieval system that indexes personal photos, documents, receipts, and scans entirely on-device with zero cloud offloading. The system processes multimodal personal data via a split-branch architecture: OCR with inverted indexing (BM25) for documents, and vision-language vector embeddings for scenes. This report details progress on corpus construction, baseline architecture, model selection for desktop simulation vs. edge deployment, empirical benchmark evaluations, and the next-stage roadmap.

2 Labeled Dataset Creation

Personal device storage exhibits extreme modality imbalance: camera photos, noisy document scans, identity cards, financial records, and digital receipts. To simulate realistic device storage, a balanced, labeled corpus of **629 images** (627 unique indexed; 2 duplicate receipts resolved) was constructed across **31 subtypes** (14 document, 17 scene):

- **Document Bucket (289 items, 14 categories):** Aadhaar (24), PAN (21), DL (22), Passport (22), Bank Statement (20), Passbook (20), Credit Card (20), Debit Card (20), Bill (20), Receipt (20), Report Card (20), Book (20), IAM Handwritten Page (20), and General Doc (20).
- **Scene Bucket (340 items, 17 categories, 20 each):** Beach, Mountain, Trees, Food, Fitness, Wedding, Party, Travel, Planes, Transit Hub, Mall, College, Clothes, Art, Outdoor Games, Vehicle, and X-ray.

2.1 Sourcing and Annotation Methodology

The dataset combines authentic open benchmarks with privacy-preserving synthetic specimens:

1. **Public Benchmarks (449 items):** SROIE (ICDAR 2019) for receipts with upstream word-level OCR ground truth; IAM Handwriting lines assembled into full pages; SUN397 dataset (via HF datasets server) across 10 scene categories; and Wikimedia Commons public identity card specimens.
2. **Privacy-Preserving Synthetics (180 items):** Realistically rendered Indian identity cards, passbooks, and statements preserving true typography and layout without exposing real PII.
3. **Ground Truth & Judgments:** A master manifest (`manifest.csv`) tracks SHA-256 hashes, source URLs, licenses, dimensions, and subtype labels. A test suite of **98 user queries** (`queries.jsonl`) covering all 31 subtypes was expanded into **2,183 graded relevance judgments** (`qrels.csv`).

3 Models Used for PC Simulation

The desktop simulation verifies algorithmic plumbing and provides a rigorous performance baseline:

- **Vision-Language Representation (CLIP):** `ViT-B-32` pre-trained on `laion2b_s34b_b79k` via OpenCLIP. Reused dual-purpose: zero-shot document/scene routing and 512-dimensional float32 scene embeddings.
- **Text Recognition (OCR):** Pure **Tesseract OCR (v5.x)** with English language pack (`eng`). Extracts bounding boxes, confidences (≥ 30), and normalized text tokens without external post-processing pipelines.
- **Inverted Index & BM25 Ranking: SQLite FTS5**. Integrates full-text inverted indexing with native Okapi BM25 scoring and column weighting (1× OCR body text, 5× document subtype tags).
- **Vector Store:** Exact cosine similarity matrix multiplication over L2-normalized vectors stored in SQLite.

4 Finalized Models for Edge Device (Android)

For edge deployment (Android smartphone), model selections must satisfy strict memory limits (< 150 MB), zero cloud dependencies, low latency, and battery efficiency:

- **On-Device Multimodal Encoder: MobileCLIP** (`MobileCLIP-S0` / `S2`) or quantized TFLite/ONNX-Runtime MobileCLIP. Replaces ViT-B-32 with a lightweight footprint (< 40 MB), delivering 4× faster NPU inference with minimal zero-shot degradation.
- **On-Device OCR Engine: Google ML Kit Text Recognition v2**. Shipped natively via Android Play Services; executes on-device without model bundling bloat, offering hardware-accelerated Latin and Devanagari recognition at < 100 ms/frame.
- **Storage & FTS Index:** Native **Android SQLite FTS5**. Zero dependency overhead; exact schema and BM25 parity with desktop simulation.
- **Vector Search Seam:** Lightweight BLOB vector scan or **usearch / ObjectBox** embedded vector store for sub-millisecond on-device similarity search.

5 Baseline Architecture

The baseline implements a single-shot, un-fused pipeline as illustrated in Figure 1:

1. **Stage 0 (Ingest):** Computes SHA-256 for de-duplication, parses EXIF metadata, and flags screenshots via aspect ratio heuristics.
2. **Stage 1 (Classification):** Zero-shot scoring against document vs. scene prompt prototypes. Margin < 0.05 hedges items into *both* branches.
3. **Stage 2a (Document Branch):** Pure Tesseract OCR extracts text blocks → rule-based subtype tagging → indexed into SQLite FTS5.
4. **Stage 2b (Scene Branch):** 512-dim visual embeddings stored in SQLite vector store.
5. **Stage 3 (Query Routing):** Lexical rules classify intents ($> 95\%$ accuracy) with CLIP semantic fallback, directing

queries to FTS5 (BM25) or Vector Search.

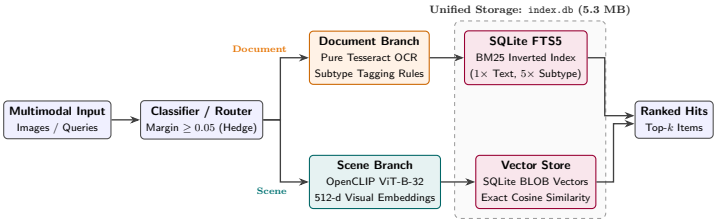


Figure 1: Pixarch Dual-Branch Baseline Architecture.

6 Results & Performance Benchmarks

Platform Note: Currently, all benchmark results were conducted on **PC simulation** (desktop environment). The next stage of development will focus on implementing and evaluating these models directly on an **edge device** (Android smartphone).

6.1 Information Retrieval Performance

Evaluated across 98 queries over 627 indexed items (retrieval depth 107). Table 1 compares the Routed Baseline against single-model comparators.

System	P@1	P@5	MRR@10	nDCG@10	R-prec	R@10
Routed Baseline	**0.773**	**0.781**	**0.803**	**0.773**	**0.693**	**0.369**
CLIP-only	0.629	0.596	0.644	0.580	0.478	0.278
BM25-only	0.361	0.355	0.378	0.349	0.312	0.157

Table 1: Retrieval effectiveness vs. single-model comparators (98 queries).

Routing Value & Recall Ceilings: Routing beats both single-model comparators overall (+0.144 P@1 over CLIP-only; +0.412 over BM25-only). Single-model systems collapse on opposing domains: BM25 scores 0.020 on scenes; CLIP scores 0.370 on documents. Macro-averaged P@1 across buckets highlights routing robustness: Routed (**0.772**) tracks the upper bound, while CLIP-only drops to 0.616 and BM25-only to 0.379. With ∼20 relevant items per subtype, Recall@10 caps at 0.480; the baseline achieves **77.0% of achievable recall** (0.369/0.480) and **0.693** R-precision.

6.2 Ingestion Cost & Throughput

Full corpus indexing of 627 items completed in 352.4 s on PC (**1.78 images/sec** end-to-end throughput).

Stage	Items	Time (s)	ms/image	Throughput
0. Ingest	627	18.2	29.1 ms	34.41 img/s
1. CLIP Encode	627	27.1	43.2 ms	23.14 img/s
1b. Routing/Classify	627	16.9	26.9 ms	37.22 img/s
2b. Embed Store	627	0.1	0.2 ms	4209.58 img/s
2a. Tesseract OCR	393	290.0	738.0 ms	1.35 img/s
End-to-End Pipeline	**627**	**352.4**	**562.0 ms**	**1.78 img/s**

Table 2: Ingestion pipeline stage timings and throughput.

Throughput Bottleneck: OCR constitutes **82%** of total indexing time, making documents ∼11× more expensive than scenes (scene: 10.06 img/s; doc: 0.90 img/s). A 10,000-item gallery projects to 1.56 hours indexing on desktop GPU.

6.3 Query Latency

Evaluated across warm sequential benchmark queries ($n = 98$):

- **Routed Document:** Mean **4.1 ms**, p50 **1.6 ms**, p95 **8.5 ms**, max 46.9 ms.
- **Routed Scene:** Mean **11.9 ms**, p50 **12.4 ms**, p95 **15.2 ms**, max 63.3 ms.
- **Overall Queries:** Mean **8.0 ms**, p50 **6.8 ms**, p95 **13.5 ms**, max 63.3 ms.

6.4 Module Benchmarks & Storage

- **Bucket Classification (CLIP Zero-Shot): 98.6%** accuracy (618/627 correct; 8 scene → doc, 1 doc → scene).
- **Query Routing Accuracy: 95.9%** (94/98 correct; 3 misrouted are X-rays, 1 intentionally ambiguous).
- **Subtype Tagging (`doctype.py`): 70.7%** overall (100% on synthetic; 21.5% on photographed documents).
- **Tesseract OCR Accuracy (258 items):** Overall CER = **0.471**, WER = **0.596**; excluding bill: CER = **0.420**, WER = **0.527** (synthetic CER 0.418 vs. fetched 0.592).
- **Storage Footprint:** `index.db` is **5.3 MB** (≈**8.5 KB/image**; embeddings 48.3%, OCR blocks 28.9%, inverted index 6.7%). Projected 10k items gallery is **85 MB**.

7 Next Stage Development & Roadmap

Building upon the established PC baseline, the upcoming phase focuses on edge porting, quality refinement, and interaction extensions:

1. **Mobile Device Implementation:** Deploying the full pipeline on Android using Google ML Kit Text Recognition v2 and quantized MobileCLIP (FP16/INT8), optimizing on-device memory footprints and battery efficiency.
2. **Pre-OCR Image Processing:** Introducing adaptive binarization, skew correction, and contrast normalization prior to OCR to enhance character recognition on noisy and mobile-captured scans.
3. **Document Semantic Search & Tradeoffs:** Integrating dense neural document embeddings alongside BM25, measuring on-device latency vs. retrieval recall tradeoffs.
4. **Query Rewriting:** Implementing lightweight local query expansion to resolve ambiguous terms and improve intent routing.
5. **OCR Correction via `ocrfixr`:** Evaluating the feasibility and latency overhead of running token repair (`ocrfixr`) on mobile hardware to fix corrupted OCR text.
6. **Image-Conditioned Multimodal Search:** (Time permitting) Supporting image-plus-query search (e.g., *"find images with similar background"*).

8 Source Code Repository

The complete reproducible codebase, configuration files, manifests, test suites, and evaluation scripts are available at:

https://github.com/shivam-git-acc/Pixarch